

Work environment preparation

PGSCE – Programação Avançada de Sistemas Embebidos

António Barros

Departamento de Engenharia Informática

Disclaimer: The instructions provided in this session are intended for educational purposes and may not cover all aspects of setting up a FreeRTOS development environment. It is recommended to refer to official documentation and resources for the most up-to-date and comprehensive information.

AI Disclosure: The following steps were suggested by the Gemini LLM, but have been manually verified and corrected to ensure they are accurate and functional. Additionally, some parts of the text were automatically suggested by the CoPilot in VSCode, but have been reviewed and edited to ensure clarity and precision of the instructions.

Setting Up a Native Development Environment on macOS

Installing the Required Software with Homebrew

Software Environment Setup

Requirement Overview & Configuration Methods

- **Manual Installation:** The installation of the necessary software environment does not strictly necessitate the use of Homebrew. All required components can be configured manually through official installers or binary distributions.
- **Homebrew Integration:** Whilst optional, the adoption of Homebrew is recommended. Its utilisation facilitates the installation process by automating dependency management and ensuring a standardised setup.

Step 1: Install Homebrew

Homebrew is a popular package manager for macOS that simplifies the installation of software and development tools.

To install Homebrew, open the Terminal application and run the following command:

Install Homebrew

```
% /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Verify the installation by running:

Verify Homebrew installation

```
% brew --version
```

Step 1: Install Homebrew

After installing Homebrew, you need to add it to your system's PATH environment variable to ensure that you can use the applications installed with Homebrew from the Terminal.

Run the following commands in the Terminal:

Add Homebrew directory to PATH

```
% echo >> ${HOME}/.zprofile  
% echo 'eval "$(/usr/local/bin/brew shellenv zsh)"' >> ${HOME}/.zprofile  
% eval "$(/usr/local/bin/brew shellenv zsh)"
```

Step 2: Install the ARM GNU toolchain

After installing Homebrew, you can use it to install the ARM GNU toolchain, which is necessary for cross-compiling FreeRTOS applications for ARM Cortex-M microcontrollers.

Run the following command in the Terminal:

Install ARM GNU toolchain

```
% brew install --cask gcc-arm-embedded
```

This command will download and install the ARM GNU toolchain on your macOS Intel system.

You can download the ARM GNU toolchain for macOS Intel from the official Arm Developer website, instead:

<https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads>

Step 2: Install the ARM GNU toolchain

After installing the ARM GNU toolchain, you can verify that it was installed correctly by checking the version of the compiler and the sysroot path.

Run the following commands in the Terminal:

Verify ARM GNU toolchain installation

```
% arm-none-eabi-gcc --version
```

```
% arm-none-eabi-gcc -print-sysroot
```

If the installation was successful, you should see output indicating the version of the ARM GNU toolchain that is installed on your system and the path to the sysroot directory, which contains the necessary libraries and headers for cross-compiling FreeRTOS applications.

Step 3: Install QEMU

QEMU is an open-source emulator that can be used to run and test FreeRTOS applications on a virtual ARM Cortex-M microcontroller.

To install QEMU, run the following command in the Terminal:

Install QEMU

```
% brew install qemu
```

This command will download and install QEMU on your macOS Intel system, allowing you to emulate ARM Cortex-M microcontrollers for testing your FreeRTOS applications. Verify the installation by running:

Verify QEMU installation

```
% qemu-system-arm --version
```

Step 4: Install utility tools Cmake (optional)

CMake is a cross-platform build system that can be used to manage the build process of your FreeRTOS applications.

To install CMake, run the following command in the Terminal:

Install CMake

```
% brew install cmake
```

This command will download and install CMake on your macOS Intel system, allowing you to configure and build your FreeRTOS applications more efficiently. Verify the installation by running:

Verify CMake installation

```
% cmake --version
```

Step 5: Obtain the FreeRTOS Kernel

You can obtain the FreeRTOS kernel source code from the official FreeRTOS website or from its GitHub repository.

To clone the FreeRTOS kernel from GitHub, run the following command in the Terminal from the directory where you want to store the FreeRTOS source code:

Clone FreeRTOS kernel

```
% git clone https://github.com/FreeRTOS/FreeRTOS.git --recurse-submodules
```

This command will download the FreeRTOS kernel source code to your local machine, allowing you to start developing and testing your FreeRTOS applications.

Step 6: Build the FreeRTOS application

To build your FreeRTOS application, navigate to the directory containing your Makefile file – e.g. /build/gcc/ – and run the make command in the Terminal:

Build FreeRTOS application

```
% make
```

This command will compile the FreeRTOS application using Make. If the build is successful, you should see an executable file (e.g., FreeRTOS_App) in the build directory.

Step 7: Emulate with QEMU

After building your FreeRTOS application, you can use QEMU to emulate an ARM Cortex-M microcontroller and run your application.

Run the following command in the Terminal to start the QEMU emulator:

```
% qemu-system-arm -machine mps2-an385 -nographic -kernel RTOSDemo.out
```

This command will start the QEMU emulator with the mps2-an385 board configuration and load your RTOSDemo.out binary for execution. You should see the output from your application in the QEMU console, allowing you to test and debug your FreeRTOS application in a virtual environment.

Installing a native development environment: Linux

Step 1: Install Essential Build Tools

On Linux systems, you can install essential build tools using the package manager. For example, on Ubuntu or Debian-based systems, you can run:

Install Essential Build Tools

```
$ sudo apt update  
$ sudo apt install build-essential cmake ninja-build gcc-arm-none-eabi qemu-system-arm gdb-multiarch
```

Step 2: Obtain the FreeRTOS Kernel

You can obtain the FreeRTOS kernel source code from the official FreeRTOS website or from its GitHub repository.

To clone the FreeRTOS kernel from GitHub, run the following command in the Terminal from the directory where you want to store the FreeRTOS source code:

Clone FreeRTOS kernel

```
% git clone https://github.com/FreeRTOS/FreeRTOS.git --recurse-submodules
```

This command will download the FreeRTOS kernel source code to your local machine, allowing you to start developing and testing your FreeRTOS applications.

Step 3: Build the FreeRTOS application

To build your FreeRTOS application, navigate to the directory containing your Makefile file – e.g. /build/gcc/ – and run the make command in the Terminal:

Build FreeRTOS application

```
% make
```

This command will compile the FreeRTOS application using Make. If the build is successful, you should see an executable file (e.g., FreeRTOS_App) in the build directory.

Step 4: Emulate with QEMU

To emulate your FreeRTOS application using QEMU, run the following command:

```
$ qemu-system-arm -machine mps2-an385 -nographic -kernel RTOSDemo.out
```

This command starts the QEMU emulator with the mps2-an385 machine profile and loads your RTOSDemo.out binary for execution.

Setting Up a Project (generated by Gemini)

Set Up Your Development Environment

After installing the necessary tools, you can set up your development environment for FreeRTOS. This typically involves creating a project directory, configuring CMake for your FreeRTOS application, and building the project.

```
my_freertos_project/  
+-- build/           # Output directory  
+-- src/             # Source code (.c and .h)  
+-- FreeRTOS/        # Operating system source  
`-- CMakeLists.txt   # Build script
```

Configure the CMakeLists.txt (MPS2 QEMU)

To build for the MPS2 Cortex-M3 platform on QEMU, your CMakeLists.txt must specify the M3 architecture and the corresponding FreeRTOS port.

CMakeLists.txt (generated by Gemini)

```
cmake_minimum_required(VERSION 3.16)
project(FreeRTOS_MPS2_Demo C ASM)
# 1. Target System (Bare Metal ARM)
set(CMAKE_SYSTEM_NAME Generic)
set(CMAKE_SYSTEM_PROCESSOR arm)
# 2. Architecture Flags for MPS2 (Cortex-M3, no FPU)
add_compile_options(
    -mcpu=cortex-m3 -mthumb -mfloat-abi=soft
    -ffunction-sections -fdata-sections -g -O0
)
# 3. Linker Script for MPS2 QEMU Memory Map
set(LINKER_SCRIPT ${CMAKE_SOURCE_DIR}/gcc/mps2_m3.ld)
add_link_options(-T ${LINKER_SCRIPT} --specs=nano.specs -Wl,--gc-sections)
# 4. FreeRTOS Port: GCC/ARM_CM3
add_executable(${PROJECT_NAME} main.c startup_gcc.c)
target_include_directories(${PROJECT_NAME} PRIVATE
    FreeRTOS/Source/include
    FreeRTOS/Source/portable/GCC/ARM_CM3
    . # Path to FreeRTOSConfig.h
)
```

Build the FreeRTOS application (generated by Gemini)

To build your FreeRTOS application, navigate to the directory containing your CMakeLists.txt file and run the following commands in the Terminal:

Build FreeRTOS application

```
$ mkdir build  
$ cd build  
$ cmake ..  
$ make
```

If the build is successful, you should see an executable file (e.g., RTOSDemo.out) in the output directory.